Структура табличных файлов CSV и потоковое чтение

Устройство текстовых таблиц с разделителями и чтение строковых матриц встроенным модулем csv

Когда программа обменивается таблицами с внешними системами (Excel, 1С, реляционные базы данных), обычного метода `.split(";")` становится недостаточно. Если внутри названия товара встретится точка с запятой или перенос строки, простая разбивка разорвет ячейку на части и сдвинет все столбцы. Для надежного разбора таких таблиц в Python встроен модуль `csv` (Comma-Separated Values).

Устройство формата CSV и базовые инструменты чтения

ШАПКА И СТРОКИ	**Логическая разметка таблицы:** первая строка файла почти всегда содержит названия колонок (заголовок), а каждая последующая строка описывает одну запись с тем же числом полей.
`csv.reader(file)`	**Потоковый парсер строк:** читает открытый файл построчно и автоматически возвращает каждую запись в виде готового списка строк Python `['ячейка_1', 'ячейка_2']`.
`next(reader)`	**Сдвиг курсора чтения:** извлекает ровно одну верхнюю строку из потока `reader`. Позволяет сохранить шапку таблицы в отдельную переменную перед основным циклом `for`.

Пример: Построчное чтение ведомости оборудования и расчет стоимости

csv_stream_reader.py — PYTHON 3

```python
import csv

# 1. Создаем демонстрационный CSV-файл на диске для последующего чтения
raw_csv = "Артикул;Наименование узла;Количество;Цена\nSW-104;Коммутатор 24p;3;42000\nRT-208;Маршрутизатор;5;18500"
with open("equipment.csv", "w", encoding="utf-8") as f:
    f.write(raw_csv)

# 2. Открываем файл для чтения с обязательным параметром newline=""
with open("equipment.csv", "r", encoding="utf-8", newline="") as file:
    reader = csv.reader(file, delimiter=";")  # Указываем разделитель столбцов ";"
    header = next(reader)                     # Извлекаем первую строку (шапку таблицы)
    print(f"Загружены колонки: {header}")

    # Перебираем оставшиеся строки данных (каждая запись row — это список из 4 строк)
    for row in reader:
        sku, name = row[0], row[1]           # Индексы 0 и 1: строковые артикул и название
        qty, price = int(row[2]), int(row[3]) # Индексы 2 и 3: переводим строки в числа int
        print(f"Узел [{sku}] {name} | Сумма партии: {qty * price} руб.")
```

ВЫВОД В ТЕРМИНАЛ: ПОСТРОЧНЫЙ РАЗБОР CSV-ТАБЛИЦЫ — SUCCESS

```
Загружены колонки: ['Артикул', 'Наименование узла', 'Количество', 'Цена']
Узел [SW-104] Коммутатор 24p | Сумма партии: 126000 руб.
Узел [RT-208] Маршрутизатор | Сумма партии: 92500 руб.
```

Все ячейки читаются как текст: Модуль `csv` никогда не угадывает типы данных автоматически. Даже если в таблице записано число `42000`, парсер вернет строку `'42000'`. Перед умножением или сложением всегда выполняйте явное приведение типов `int()` или `float()`.

Экспорт списков в CSV и правила экранирования

Запись двумерных коллекций на диск через csv.writer и защита структуры от технических символов

При выгрузке отчетов из памяти Python на диск важно соблюдать региональные стандарты табличных процессоров. В русскоязычной локали Microsoft Excel символом-разделителем целой и дробной части числа выступает запятая (например, `23,5`), поэтому для разделения столбцов используется точка с запятой `delimiter=";"`.

Инструменты записи таблиц и управления кавычками

`writer.writerow(row)`	**Запись одной строки:** принимает один плоский список `['A1', 'B1', 100]`, преобразует все его элементы в строки и записывает одну горизонтальную линию таблицы.
`writer.writerows(list)`	**Пакетная выгрузка матрицы:** принимает вложенный список списков (всю таблицу целиком) и записывает все строки на диск за один вызов без цикла `for`.
`newline=""`	**Защита от пустых строк:** обязательный аргумент функции `open()` при работе с CSV. Без него на ОС Windows после каждой строки данных будет появляться пустая строка-разрыв.

Пример: Формирование отчета с автоматическим экранированием спецсимволов

csv_export_writer.py — PYTHON 3

```python
import csv

# Обратите внимание: описание первого кабеля содержит точку с запятой внутри текста!
shipments = [
    ["Код партии", "Описание груза", "Вес (кг)"],
    ["LOT-01", "Кабель витая пара; бухта 305м", 11.4],
    ["LOT-02", "Шкаф телекоммуникационный 42U", 68.0]
]

# 1. Открываем файл для перезаписи ("w") с отключенной трансляцией переводов строк
with open("shipments.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file, delimiter=";")  # Сам обернет проблемные ячейки в кавычки "..."
    writer.writerows(shipments)               # Записываем двумерный список одной командой
    writer.writerow(["LOT-03", "Комплект крепежа; гайки M6", 2.1]) # Дописываем строку

# 2. Прочитаем сырой текст файла, чтобы увидеть, как csv.writer защитил точку с запятой
with open("shipments.csv", "r", encoding="utf-8") as raw_file:
    print(raw_file.read().strip())
```

ВЫВОД В ТЕРМИНАЛ: СОДЕРЖИМОЕ ФАЙЛА С АВТОМАТИЧЕСКИМИ КАВЫЧКАМИ — SUCCESS

```
Код партии;Описание груза;Вес (кг)
LOT-01;"Кабель витая пара; бухта 305м";11.4
LOT-02;Шкаф телекоммуникационный 42U;68.0
LOT-03;"Комплект крепежа; гайки M6";2.1
```

Почему нельзя собирать CSV через f-строки: Если бы мы записали строку вручную через `f"{code};{desc};{weight}\n"`, точка с запятой внутри фразы *«Кабель витая пара; бухта 305м»* разорвала бы столбец на две части. Объект `csv.writer` автоматически обнаружил конфликт и изолировал текст кавычками.

Работа с именованными полями: DictReader и DictWriter

Автоматическое связывание заголовков столбцов со словарями Python для повышения читаемости кода

При работе с таблицами из 15–20 столбцов обращение по числовым индексам `row[0]`, `row[7]` приводит к путанице. Кроме того, если внешняя система поменяет местами два столбца в файле, программа начнет читать неверные данные. Классы `DictReader` и `DictWriter` решают эту проблему, превращая каждую строку таблицы в словарь.

Инструменты словарного отображения CSV-таблиц

`csv.DictReader(file)`	**Чтение в словари:** автоматически берет первую строку файла как список ключей, а при переборе в цикле `for` возвращает каждую запись как словарь `{'колонка': 'значение'}`.
`fieldnames=[...]`	**Схема столбцов для записи:** обязательный список названий колонок, передаваемый в `DictWriter`. Определяет строгий порядок столбцов в создаваемом файле.
`writer.writeheader()`	**Генерация шапки:** метод `DictWriter`, который автоматически записывает верхнюю строку с названиями столбцов, указанными в параметре `fieldnames`.

Пример: Экспорт и чтение карточек серверов по именам столбцов

csv_dict_engine.py — PYTHON 3

```python
import csv

# База данных в памяти представлена как список словарей с одинаковыми ключами
cluster_nodes = [
    {"hostname": "srv-auth-01", "ip": "10.20.1.10", "cpu_load": 45},
    {"hostname": "srv-db-02",   "ip": "10.20.1.15", "cpu_load": 88}
]
columns = ["hostname", "ip", "cpu_load"]  # Задаем строгий порядок следования колонок

# Шаг 1: Сохраняем список словарей в CSV-файл через DictWriter
with open("nodes.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.DictWriter(file, fieldnames=columns, delimiter=";")
    writer.writeheader()               # Записываем строку заголовков: hostname;ip;cpu_load
    writer.writerows(cluster_nodes)    # Выгружаем все словари из списка

# Шаг 2: Читаем файл обратно через DictReader (вызов next() больше не нужен!)
with open("nodes.csv", "r", encoding="utf-8", newline="") as file:
    for node in csv.DictReader(file, delimiter=";"):
        print(f"Сервер: {node['hostname']} | IP: {node['ip']} | ЦП: {node['cpu_load']}%")
```

ВЫВОД В ТЕРМИНАЛ: ДОСТУП К КОЛОНКАМ ПО ИМЕНАМ КЛЮЧЕЙ — SUCCESS

```
Сервер: srv-auth-01 | IP: 10.20.1.10 | ЦП: 45%
Сервер: srv-db-02 | IP: 10.20.1.15 | ЦП: 88%
```

Переход к следующей теме (от чтения файлов к полному циклу обработки коллекций):

Мы изучили инструменты чтения и записи табличных файлов. Однако в реальных задачах программирования данные никогда не перекладываются из файла в файл без изменений: перед сохранением результата их необходимо очистить от мусора, отфильтровать, отсортировать и обогатить расчетами в памяти.

Архитектура конвейера данных: загрузка и очистка

Построение алгоритма извлечения сырых строк из файла, нормализации текста и приведения типов

Любая прикладная программа обработки данных строится по принципу конвейера ETL (*Extract, Transform, Load* — Извлечение, Преобразование, Загрузка). Первая фаза этого алгоритма отвечает за чтение «грязных» данных с диска, удаление случайных пробелов, фильтрацию пустых строк и перевод текстовых полей в числа.

Этапы первичной нормализации данных при загрузке в список

ШАГ 1: ОЧИСТКА СТРОК	**Удаление невидимых символов:** применение методов `.strip()` и `.replace(" ", "")` убирает лишние пробелы по краям ячеек и внутри числовых значений.
ШАГ 2: ОТСЕВ ПУСТОТ	**Фильтрация битых записей:** проверка `if not line: continue` пропускает пустые строки в конце файла, которые иначе вызвали бы сбой при распаковке.
ШАГ 3: ТИПИЗАЦИЯ	**Конвертация и обогащение:** преобразование строк в `int` / `float` и немедленный расчет дополнительных полей (например, стоимости с учетом налога или скидки).

Пример: Загрузка и нормализация журнала датчиков из текстового файла

etl_extract_clean.py — PYTHON 3

```python
# 1. Запишем на диск "сырой" лог с лишними пробелами, разным регистром и пустой строкой
dirty_log = "  sens-01 ; Цех сборки ; 24.5 \nSENS-02 ; Склад ГСМ ; 18.0\n\n  sens-03 ; Серверная ; 29.8 "
with open("raw_sensors.txt", "w", encoding="utf-8") as f:
    f.write(dirty_log)

# 2. Читаем файл, очищаем данные и собираем чистую коллекцию словарей
clean_sensors = []
with open("raw_sensors.txt", "r", encoding="utf-8") as file:
    for line in file:
        stripped = line.strip()            # Убираем пробелы по краям и перенос строки \n
        if not stripped:                   # Если строка оказалась пустой — пропускаем ее
            continue
        raw_id, raw_zone, raw_temp = stripped.split(";") # Разбиваем строку на 3 части
        clean_sensors.append({
            "id": raw_id.strip().upper(),   # Приводим код датчика к верхнему регистру
            "zone": raw_zone.strip(),       # Очищаем название помещения
            "temp": float(raw_temp.strip()) # Переводим температуру в дробное число float
        })

print(f"Очищено записей: {len(clean_sensors)} | Данные: {clean_sensors}")
```

ВЫВОД В ТЕРМИНАЛ: РЕЗУЛЬТАТ НОРМАЛИЗАЦИИ ДАННЫХ — SUCCESS

```
Очищено записей: 3 | Данные: [{'id': 'SENS-01', 'zone': 'Цех сборки', 'temp': 24.5}, {'id': 'SENS-02',
'zone': 'Склад ГСМ', 'temp': 18.0}, {'id': 'SENS-03', 'zone': 'Серверная', 'temp': 29.8}]
```

Правило санитарной границы: Никогда не пускайте неочищенные строки в расчетную часть программы. Все операции `.strip()`, `.upper()` и приведение типов должны выполняться сразу в момент чтения файла, чтобы остальной код работал со строго типизированными коллекциями.

Преобразование, фильтрация и сортировка в списках

Упорядочивание списков словарей по ключу и алгоритмический отбор записей в оперативной памяти

Когда данные загружены из файла в список словарей, программа переходит ко второму этапу — аналитической трансформации. Нам требуется отобрать записи по нескольким критериям, рассчитать новые показатели и выстроить элементы строго по возрастанию или убыванию целевого параметра.

Инструменты сортировки и трансформации коллекций

`list.sort(key=...)`	**Сортировка списка на месте:** перестраивает текущий список. Так как Python не знает, по какому именно полю сравнивать словари, в параметр `key` передается функция выбора поля.
`lambda item: item["k"]`	**Анонимная функция-селектор:** компактная однострочная инструкция, которая указывает методу `.sort()` брать для сравнения значение конкретного ключа `"k"` из каждого словаря.
`reverse=True`	**Управление направлением:** по умолчанию сортировка идет по возрастанию (от меньшего к большему). Флаг `reverse=True` разворачивает порядок по убыванию (лидеры сверху).

Пример: Расчет премии сотрудников, фильтрация и построение рейтинга

collection_transform_sort.py — PYTHON 3

```python
staff = [
    {"name": "Иванов А.С.",  "kpi": 115, "base_salary": 80000},
    {"name": "Петрова Е.В.", "kpi": 92,  "base_salary": 85000},
    {"name": "Сидоров Д.К.", "kpi": 140, "base_salary": 90000},
    {"name": "Смирнов И.О.", "kpi": 125, "base_salary": 78000}
]

# Шаг 1: Отбираем сотрудников с выполненным планом (KPI >= 100) и начисляем 20% премии
bonus_list = []
for emp in staff:
    if emp["kpi"] >= 100:
        emp["total_pay"] = int(emp["base_salary"] * 1.20)  # Добавляем ключ итоговой выплаты
        bonus_list.append(emp)

# Шаг 2: Сортируем премиальный список по убыванию итоговой выплаты ("total_pay")
bonus_list.sort(key=lambda x: x["total_pay"], reverse=True)

for rank, person in enumerate(bonus_list, start=1):
    print(f"Место #{rank}: {person['name']} | KPI: {person['kpi']}% | К выплате: {person['total_pay']}
руб.")
```

ВЫВОД В ТЕРМИНАЛ: РЕЙТИНГ СОТРУДНИКОВ ПОСЛЕ ФИЛЬТРАЦИИ И СОРТИРОВКИ — SUCCESS

```
Место #1: Сидоров Д.К. | KPI: 140% | К выплате: 108000 руб.
Место #2: Иванов А.С. | KPI: 115% | К выплате: 96000 руб.
Место #3: Смирнов И.О. | KPI: 125% | К выплате: 93600 руб.
```

Разница между .sort() и sorted(): Метод `bonus_list.sort()` изменяет сам исходный список в памяти и возвращает `None`. Если вам нужно сохранить исходный порядок записей нетронутым, используйте функцию `new_list = sorted(bonus_list, key=...)`, которая создаст новую отсортированную копию.

Группировка коллекций и сохранение сводного отчета

Агрегация записей по категориям с помощью словарей-накопителей и выгрузка итогов на диск

Заключительный этап алгоритма работы с коллекциями и файлами — сведение сотен детальных строк в компактную итоговую таблицу (агрегация по группам) и запись результата в новый файл. Для подсчета сумм по отделам, категориям или городам идеально подходит вспомогательный словарь-аккумулятор.

Паттерн группировки через словарь и фиксация результата

СЛОВАРЬ-НАКОПИТЕЛЬ	**Схема «Категория: Сумма»:** создается пустой словарь `totals = {}`, где ключом становится название категории, а значением — накапливаемая сумма показателей.
`totals.get(cat, 0)`	**Безопасное накопление в одну строку:** выражение `totals[cat] = totals.get(cat, 0) + val` берет текущую сумму категории (или `0`, если она встретилась впервые) и прибавляет число.
ФИНАЛЬНАЯ ВЫГРУЗКА	**Экспорт агрегата:** преобразование собранного словаря `totals.items()` обратно в строки CSV-отчета и сохранение на жесткий диск через менеджер `with`.

Пример: Полный цикл от чтения транзакций до выгрузки сводного CSV

full_etl_pipeline.py — PYTHON 3

```python
import csv

# Исходный список транзакций, полученный после чтения операционного журнала
orders = [
    {"category": "Серверы", "amount": 240000}, {"category": "Кабели", "amount": 15000},
    {"category": "Серверы", "amount": 180000}, {"category": "Кабели", "amount": 9500},
    {"category": "Лицензии", "amount": 65000}
]

# Шаг 1: Агрегируем выручку и количество заказов по каждой категории
summary = {}
for row in orders:
    cat = row["category"]
    if cat not in summary:
        summary[cat] = {"count": 0, "total_rub": 0} # Инициализируем новую категорию
    summary[cat]["count"] += 1
    summary[cat]["total_rub"] += row["amount"]

# Шаг 2: Сохраняем сгруппированный результат в итоговый CSV-файл
with open("category_report.csv", "w", encoding="utf-8", newline="") as file:
    writer = csv.writer(file, delimiter=";")
    writer.writerow(["Категория", "Число сделок", "Итоговая выручка"])
    for cat_name, stats in summary.items():
        writer.writerow([cat_name, stats["count"], stats["total_rub"]])
        print(f"Направление '{cat_name}': сделок {stats['count']} на сумму {stats['total_rub']} руб.")
```

ВЫВОД В ТЕРМИНАЛ: СВОДНАЯ АНАЛИТИКА И ЗАПИСЬ ОТЧЕТА — SUCCESS

```
Направление 'Серверы': сделок 2 на сумму 420000 руб.
Направление 'Кабели': сделок 2 на сумму 24500 руб.
Направление 'Лицензии': сделок 1 на сумму 65000 руб.
```

Переход к следующей теме (от идеальных алгоритмов к защите от реальных сбоев):

В разобранных примерах файлы всегда существовали на диске, а внутри числовых колонок лежали корректные цифры. В промышленной эксплуатации файл может оказаться заблокирован, удален или поврежден. Чтобы программа не завершалась аварийно, необходимо внедрить перехват исключений и контроль целостности.

Обработка типичных ошибок доступа к файлам

Перехват исключений файловой системы и проблем кодировки через конструкцию try-except

При обращении к жесткому диску или сетевой папке программа зависит от операционной системы. Если запрошенный файл был перемещен или удален пользователем, вызов `open()` немедленно обрушит весь скрипт с фатальной ошибкой `FileNotFoundError`.

unprotected_open_crash.py — PYTHON 3

```python
# Опасное чтение без защиты: если файла "missing_db.csv" нет на диске, программа упадет!
with open("missing_db.csv", "r", encoding="utf-8") as file:
    data = file.read()
```

ВЫВОД В ТЕРМИНАЛ (АВАРИЙНЫЙ СБОЙ ПРОГРАММЫ): CRASH: FILE NOT FOUND

```
Traceback (most recent call last):
  File "unprotected_open_crash.py", line 2, in <module>
    with open("missing_db.csv", "r", encoding="utf-8") as file:
FileNotFoundError: [Errno 2] No such file or directory: 'missing_db.csv'
```

Классификация файловых сбоев и защитный блок try-except

`FileNotFoundError`	**Файл отсутствует:** неверно указано имя файла или путь к папке при открытии в режиме чтения `"r"`.
`PermissionError`	**Отказ в доступе:** попытка перезаписать CSV-файл, который в этот самый момент открыт пользователем в программе Microsoft Excel.
`UnicodeDecodeError`	**Конфликт кодировок:** попытка прочитать старый кириллический файл в кодировке Windows-1251 (`cp1251`) через стандарт `utf-8`.

Пример: Защищенная функция чтения с автоматическим резервным сценарием

safe_file_gateway.py — PYTHON 3

```python
def read_table_safely(filename):
    try:
        with open(filename, "r", encoding="utf-8") as f:
            return f.readlines()
    except FileNotFoundError:
        print(f"[ВНИМАНИЕ]: Файл '{filename}' не найден. Возвращена пустая коллекция.")
        return []
    except PermissionError:
        print(f"[ОШИБКА ДОСТУПА]: Закройте файл '{filename}' в программе Excel!")
        return []

records = read_table_safely("absent_report.csv")
print(f"Работа программы успешно продолжена | Записей в памяти: {len(records)}")
```

ВЫВОД В ТЕРМИНАЛ: ШТАТНАЯ ОБРАБОТКА ОТСУТСТВУЮЩЕГО ФАЙЛА — SUCCESS

```
[ВНИМАНИЕ]: Файл 'absent_report.csv' не найден. Возвращена пустая коллекция.
Работа программы успешно продолжена | Записей в памяти: 0
```

Валидация структуры строк и защита от битых записей

Изоляция ошибок преобразования типов (ValueError) и проверка комплектности колонок внутри цикла

Даже если сам файл успешно открылся, ошибки часто скрываются внутри отдельных строк таблицы. В числовой колонке цены может оказаться текст `"нет в наличии"`, прочерк `"-"` или строка может быть оборвана посередине. Если поместить блок `try-except` **внутрь цикла** чтения, программа сможет отбраковать одну поврежденную строку и спокойно обработать остальные тысячи записей.

Виды логических повреждений внутри табличного файла

`ValueError`	**Ошибка конвертации типа:** возникает, когда функции `int()` или `float()` передается нечисловая строка (например, `int("ошибка")` или пустая ячейка `int("")`).
`IndexError`	**Неполная строка (обрыв колонок):** если в строке меньше разделителей, чем ожидалось, обращение к `row[2]` вызовет выход за границы списка.
`КАРАНТИН ЗАПИСЕЙ`	**Журнал отбраковки:** архитектурный прием, при котором поврежденные строки не просто игнорируются, а складываются в отдельный список ошибок `rejected_rows` для разбора.

Пример: Пакетный парсинг таблицы с отсевом поврежденных строк в карантин

row_level_validation.py — PYTHON 3

```python
# Имитируем строки из CSV-файла (строки #2 и #3 содержат структурные ошибки!)
incoming_rows = [
    ["PRD-01", "Монитор 27 дюймов", "24500"],
    ["PRD-02", "Клавиатура", "снято с продаж"],  # Ошибка: текст вместо цены!
    ["PRD-03", "Строка без колонки цены"],       # Ошибка: всего 2 колонки вместо 3!
    ["PRD-04", "Мышь оптическая Pro", "3200"]
]
valid_products, error_log = [], []

for line_num, row in enumerate(incoming_rows, start=1):
    try:
        if len(row) != 3:
            raise IndexError(f"Ожидалось 3 столбца, получено {len(row)}")
        price = int(row[2])  # Пытаемся перевести 3-й столбец в число (возможен ValueError)
        valid_products.append({"sku": row[0], "title": row[1], "price": price})
    except (ValueError, IndexError) as err:
        error_log.append(f"Строка #{line_num} отбракована ({err}): {row}")

print(f"Принято товаров: {len(valid_products)} | Отклонено в карантин: {len(error_log)}")
for bad_record in error_log:
    print(" ->", bad_record)
```

ВЫВОД В ТЕРМИНАЛ: ИЗОЛЯЦИЯ БИТЫХ СТРОК БЕЗ ОСТАНОВКИ ЦИКЛА — SUCCESS

```
Принято товаров: 2 | Отклонено в карантин: 2
 -> Строка #2 отбракована (invalid literal for int() with base 10: 'снято с продаж'): ['PRD-02',
'Клавиатура', 'снято с продаж']
 -> Строка #3 отбракована (Ожидалось 3 столбца, получено 2): ['PRD-03', 'Строка без колонки цены']
```

Расположение блока try-except решает всё: Если обернуть в `try` весь цикл `for` снаружи, то при первой же ошибке на строке #2 выполнение цикла прервется, и корректный товар `PRD-04` потеряется. Размещение `try` *внутри* цикла гарантирует независимую проверку каждой записи.

Тестирование целостности обмена и контрольные суммы

Сквозная верификация записи-чтения (Round-Trip), оператор assert и криптографическое хэширование SHA-256

Как убедиться, что при сохранении базы данных на диск и последующей передаче файла по сети ни одна цифра не исказилась, а типы данных при обратном чтении восстанавливаются со стопроцентной точностью? Для этого в программы обмена данными встраивают автоматические тесты целостности.

Инструменты контроля целостности информации

`ТЕСТ ROUND-TRIP`	**Проверка «Память → Диск → Память»:** исходная коллекция записывается в файл, тут же считывается обратно в новую переменную и сравнивается с оригиналом через `==`.
`assert условие`	**Инструкция самопроверки:** если логическое условие после `assert` истинно (`True`), программа идет дальше. Если ложно — немедленно вызывает ошибку `AssertionError`.
`hashlib.sha256()`	**Цифровой отпечаток файла:** алгоритм из модуля `hashlib`, вычисляющий уникальную строку из 64 символов по байтам файла. Если в файле изменится хоть одна точка, хэш изменится полностью.

Пример: Автоматическое тестирование выгрузки и сверка контрольной суммы

data_integrity_test.py — PYTHON 3

```python
import csv, hashlib

# Эталонная коллекция в памяти (поле "qty" имеет строгий целочисленный тип int)
original_data = [
    {"code": "A-100", "title": "Модуль памяти DDR5", "qty": 16},
    {"code": "B-200", "title": "Твердотельный накопитель", "qty": 8}
]

# Шаг 1: Экспортируем эталонные данные в файл "verified_stock.csv"
with open("verified_stock.csv", "w", encoding="utf-8", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=["code", "title", "qty"], delimiter=";")
    writer.writeheader()
    writer.writerows(original_data)

# Шаг 2: Вычисляем контрольную сумму SHA-256 записанного файла (режим чтения байтов "rb")
with open("verified_stock.csv", "rb") as bf:
    digest = hashlib.sha256(bf.read()).hexdigest()

# Шаг 3: Выполняем контрольное чтение (Round-Trip) с восстановлением типа int для "qty"
restored_data = []
with open("verified_stock.csv", "r", encoding="utf-8", newline="") as f:
    for r in csv.DictReader(f, delimiter=";"):
        restored_data.append({"code": r["code"], "title": r["title"], "qty": int(r["qty"])})

assert original_data == restored_data, "Ошибка: данные или типы после чтения не совпадают!"
print(f"SHA-256: {digest[:24]}... | Тест целостности успешно пройден!")
```

ВЫВОД В ТЕРМИНАЛ: РЕЗУЛЬТАТ ПРОВЕРКИ ЦЕЛОСТНОСТИ ОБМЕНА — SUCCESS

```
SHA-256: 9f4b8c201e7a6d53c812a40e... | Тест целостности успешно пройден!
```

Зачем приводить типы при Round-Trip тесте: Если при обратном чтении забыть обернуть `r["qty"]` в `int()`, в словаре останется строка `"16"` вместо числа `16`. Сравнение `original_data == restored_data` вернет `False`, и оператор `assert` мгновенно сигнализирует разработчику о потере типа данных.